

Unity MIDI Plugin — Documentation Index

This documentation describes the contents of `Assets/MIDI`, how the runtime API is structured, and how to use MIDI 1.0, MIDI 2.0 (UMP), MPE, and related transports in Unity.

NotebookLM

I've created an AI chat that registers this manual.

If you have any questions, please use this tool first. (A Google account is required.)

<https://notebooklm.google.com/notebook/c98f2512-35e0-4396-9a9d-686f662b6f4a>

Languages

- [日本語](#)
- [中文\(简体\)](#)

Contents

Getting started

- [Getting Started \(Install, Initialize, Send/Receive\)](#)
- [Build PostProcessing & Scripting Define Symbols](#)
- [Platforms & Limitations](#)

Core APIs

- [MIDI 1.0 \(MidiManager\)](#)
- [MIDI 2.0 / UMP \(Midi2Manager\)](#)
- [MPE \(MIDI Polyphonic Expression\)](#)
- [SMF / Sequencing \(jp.kshoji.midisystem\)](#)

Utilities & Editor Tools

- [Utilities \(MidiNoteUtility / MidiMessageBuilder / PitchBend / Timing\)](#)
- [Editor Tools \(Monitor / Virtual Controller / SMF Preview / Project Settings\)](#)

Gameplay Components

- [Gameplay Components \(InputMap / NoteTracker / Filter\)](#)
- [SMF Tools \(SmfPlayer / MidiRecorder / TempoMapExtractor\)](#)

Unity Ecosystem Integration

- [Unity Integration \(Timeline / Animator / Visual Scripting / Input System / Scriptable Audio / Chunity\)](#)

Genre Kits

- [Genre Kits \(Networking / Clock / Chords & Scales / Input System / Foundation\)](#)

Transports & integrations

- [Transports & Platforms](#)
- [Inter-App MIDI — Cross-platform Notes \(Android, iOS/macOS, Linux\)](#)
- [Maestro / MPTK Integration \(Virtual Devices + Adapters\)](#)

Project architecture notes

- [Virtual Devices & Event Injection](#)
- [MIDI-CI / Capability Negotiation](#)
- [Editor & Lifecycle Notes](#)
- [Embedded Third-Party Modules](#)

Samples & reference

- [Samples](#)
- [Tested Devices](#)
- [Contacts / Support](#)
- [Version History](#)

Future features (unimplemented)

The following are not included in the current release. Refer to them as a roadmap.

Already available elsewhere (do not treat as missing): pitch-bend normalize / 14-bit split (`PitchBendUtility` in [Utilities](#)); composable Channel/Device filters ([Gameplay](#)); runtime latency calibration (`MidiLatencyCalibrator` in Foundation / [Genre Kits](#)); one-way SMF → JSON export from [SMF Preview](#).

Category	Feature	Overview
Editor	Device Browser	<code>Window > MIDI > Device Browser</code> — connected device list, Vendor/Product ID, test send
Editor	Scene View Debug Overlay	Displays held notes and recent messages in the scene during Play
Editor	Latency Measurement (RTT tool)	Editor send→receive round-trip UI (for BLE / RTP-MIDI evaluation; distinct from Foundation tap calibration)
Gameplay	MPE High-level API	<code>MpeInputHandler</code> — <code>UnityEvent</code> integrating per-note expression (low-level in MPE)
Gameplay	MIDI Event Routing	Priority and exclusive control across multiple <code>MidiInputRouter</code> instances
Utilities	14-bit CC	<code>MidiCc14BitUtility</code> — assemble/decompose CC MSB/LSB pairs
Utilities	Pitch Bend → semitones	Convert 14-bit pitch bend to a configurable semitone range (normalize/split already shipped)
Utilities	UMP Parser Helper	<code>UmpParser</code> — typed decomposition of UMP word sequences
Utilities	SysEx Builder / Parser	High-level assemble and parse helpers beyond <code>MidiMessageBuilder.SystemExclusive</code> / send APIs
Utilities	SMF ↔ JSON (bidirectional runtime)	Runtime import/export both ways (Editor Preview already exports JSON one-way)
Chunity	MIDI 2.0 / UMP Mapping	32-bit velocity / per-note controller → Chuck globals (currently via MIDI 1.0 path)
Chunity	InstanceTarget API Completion	Thin wrappers over the official API surface such as <code>SetString</code> / <code>ListenForChuckEventOnce</code> / <code>Get*Array</code> / <code>RunFile</code> arguments
Chunity	Syncer / Poller	Chuck → Unity read-back via <code>Chuck*Syncer</code> / <code>EventListener</code> wrapping
Chunity	Associative Arrays, <code>*_AT</code> , VM Control	Named parameters, audio-thread write boundaries, <code>SetRunning</code> / stop Event conventions
Chunity	UGen Probe / Host Time Advancer	Waveform visualization, Unity-driven Chuck time (complementing Clock sync)
Chunity Generator	Optimization	Native pointer API, Burst, Scriptable Effect / Root Output (when needed)

Directory structure (Assets/MIDI)

- **Plugins/**

Native (and WebGL JS) plugins per platform (Android/iOS/macOS/Linux/WSA/WebGL).

- **Scripts/**

Main C# runtime:

- `MidiManager.cs` (MIDI 1.0)
- `Midi2Manager.cs` (MIDI 2.0 / UMP)
- platform plugins: `MidiPlugin.*.cs` , `Midi2Plugin.*.cs`
- event handler interfaces: `IMidi*EventHandler` , `IMidi2*EventHandler`
- MPE: `MpeManager.cs` , `IMpeEventHandler.cs`
- MIDI-Cl: `MidiCapabilityNegotiator.cs`
- virtual devices: `MidiManager.VirtualDevices.cs`

- **Scripts/Utilities/**

Domain-grouped runtime helpers (namespace `jp.kshoji.unity.midi.util`):

- `Messaging/` — `MidiOutgoingMessage` , `MidiMessageBuilder` , `PitchBendUtility` , `MidiControlSmoothing` , `AutomationInterpolation` , ...
- `MusicTheory/` — `MidiNoteUtility`
- `Smf/` — `TempoMapExtractor` , `TempoMap` , `MeasureTimeUtility` , `TupletUtility` , `SwingUtility` , `MidiMetaMessageFactory` , ...
- `Monitor/` — `MidiMonitorLogData` , `MidiMonitorFormatHelper`

- **Scripts/Editor/**

Editor extensions:

- `PostProcessBuild.cs` (build post-processing)
- `MidiMonitorWindow.cs` etc. (MIDI monitor)
- `VirtualMidiControllerWindow.cs` (virtual MIDI controller)
- `SmfPreviewWindow.cs` (SMF preview / import)
- `MidiProjectSettingsProvider.cs` (Project Settings)
- `UI/PianoKeyboardElement.cs` (includes `MidiKeyboardLogic`)

- **Resources/MidiProjectSettings.asset**

Global MIDI settings (auto-generated on first use)

- **Scripts/Gameplay/**

Gameplay components:

- `MidiInputMap.cs` / `MidiInputRouter.cs` (MIDI → UnityEvent)
- `MidiNoteTracker.cs` (held-note state management)
- `MidiFilterBase.cs` / `MidiChannelFilter.cs` / `MidiDeviceFilter.cs` (event filtering)
- `SmfPlayer.cs` / `MidiRecorder.cs` / `MidiRecordingSession.cs` (SMF playback/recording)
- `MidiClockSync.cs` / `MidiClockOutput.cs` / `MidiChordDetector.cs` (Clock sync / chord detection)
- `MidiCcProcessorBase.cs` / `MidiCcSmoother.cs` / `MidiCcButton.cs` (CC processing)

- Theory/ — ChordRecognition , MidiScaleUtility / ScaleUtility
- Scripts/Integrations/Animator/

Animator integration (MidiAnimatorDriver , MidiBlendTreeDriver)
- Scripts/Integrations/Timeline/

Timeline integration (MidiPlaybackTrack , MidiRecordTrack , markers)
- Scripts/Integrations/VisualScripting/

Visual Scripting nodes (MidiVisualScriptingBridge , event / action units)
- Scripts/Integrations/Networking/

Network MIDI sync (MidiNetworkHub , MidiNetworkClient , optional Mirror / Netcode / WSNet2 bridges, FEATURE_MIDI_NETWORK + bridge symbols)
- Scripts/Integrations/InputSystem/

Input System bridge (MidiInputSystemBridge , MidiSyntheticDevice , etc., FEATURE_INPUT_SYSTEM)
- Scripts/Integrations/ScriptableAudio/

Scriptable Audio Pipeline integration (FEATURE_SCRIPTABLE_AUDIO , Unity 6000.3+)
- Scripts/Integrations/Chunitty/

Chunitty integration (FEATURE_CHUNITY , runtime not bundled) and Scriptable Generator (FEATURE_CHUNITY_SCRIPTABLE_AUDIO)
- Scripts/Foundation/

Foundation (MidiLatencyCalibrator , MidiDeviceSelection , MidiOutputRoutingPreset , MidiFoundationUiController , etc.)
- UI/Foundation/

UXML / USS for the foundation UI
- Settings/

Shared Input System assets (e.g. MidiController.inputactions)
- Scripts/MidiSequenceAsset.cs

SMF ScriptableObject (Create > MIDI > Sequence Asset)
- Scripts/UmpSequenceAsset.cs

UMP .midi2 ScriptableObject
- Scripts/MidiProjectSettings.cs

Global MIDI settings ScriptableObject
- Scripts/midisystem/

Standard MIDI File (SMF) reader/writer + sequencing model (Sequence , Track , messages).
- Scripts/UmpSequencer/

UMP sequence utilities (clip/container read/write, sequencer, SMF↔UMP converter).
- Scripts/RTP-MIDI-for-.NET/

RTP-MIDI implementation (embedded module + its own docs).
- Scripts/UdpMidi2Discovery/

Discovery dependencies for UDP MIDI 2.0 workflows.
- Samples/

Example scenes and scripts.

- `Samples/Integrations/`
Unity ecosystem integration samples (Animator / Timeline / Visual Scripting / Networking / Input System / Chunity / Scriptable Audio).
- `Samples/Gameplay/`
Gameplay samples (Clock sync / chord & scale detection).
- `Samples/Foundation/`
Foundation samples (device selection / latency / UI shell).

Concepts & terminology

- **Deviceld**: A string identifier used throughout the API to refer to a MIDI endpoint.
- **Group**: MIDI 2.0 group index (0–15). Many MIDI 1.0 APIs accept `group` for consistency.
- **Channel**: MIDI channel (0–15). Note that some APIs use 0-based channel numbers.
- **UMP (Universal MIDI Packet)**: MIDI 2.0 packet format represented as `uint[]` words in this project.

Quick start checklist

1. Decide which feature set you need:
 - MIDI 1.0 events and sending: `MidiManager`
 - MIDI 2.0 / UMP parsing and sending: `Midi2Manager`
 - Android Inter-App MIDI: see [Inter-App MIDI](#)
 - MPE management: `MpeManager` on top of `MidiManager`
 - Bind MIDI → UnityEvent from the Inspector: `MidiInputRouter` in [Gameplay Components](#)
2. Implement one or more event handler interfaces.
3. Register handler objects with the manager.
4. Initialize the manager (or ensure it's present in the scene).
5. Test using the sample scenes in `Assets/MIDI/Samples/Scenes`.
6. During development, check input/output messages via `Window > MIDI > Monitor` ([Editor Tools](#)).